

99_risk_bottleneck_analysis

HelixLLM Full-Extension Programme — Multi-Pass Danger-Zone / Weakness / Bottleneck Analysis

Document	Stream 99 — final risk-synthesis (§11.4.92 5-pass evaluation)
Path	docs/research/ 07.2026/99_risk_analysis/ 99_risk_bottleneck_analysis.md
Revision	1
Created	2026-07-06
Author	T1/main risk-synthesis subagent
Inputs	ALL 13 prior reports (00 master + inventory + host baseline + ADR + clarifications + streams 01–12)
Method	§11.4.92 five-pass evaluation; §11.4.6 no-guessing (every claim traces to a cited stream report + its captured/cited evidence)
Track/branch	(T1/main)

Anti-bluff notice (§11.4 / §11.4.6 / §11.4.123). This is a *risk* document. It makes NO “works” claim. Every danger zone below traces to a specific prior report’s finding (captured host fact [G- *], cited web source, or read-of-source FACT). Where a prior report marked something UNCONFIRMED, it stays UNCONFIRMED here and is escalated to a must-verify-before-commit item (Pass 4). Design estimates are labelled; nothing here is a runtime proof.

0. One-paragraph orientation

The programme is an **extend**, not a greenfield build: HelixLLM already serves OpenAI + Anthropic APIs with a real llama.cpp HTTP client and a scored provider-

fallback chain; HelixAgent already ships ~45 provider adapters + an OpenAI/Anthropic/Google REST server; LLMsVerifier already has the DB schema for CONST-040 capability booleans and an endpoint-agnostic probe engine; the containers submodule already has boot/compose/health + a GPU health-check. The danger is **not** “can we build the pieces” — the pieces largely exist. The danger is concentrated in **four load-bearing seams that ALL 12 streams independently collided with**: (1) the Blackwell sm_120 toolchain, (2) rootless GPU passthrough that **does not yet exist on this host**, (3) **one 32 GB card** asked to host ~10 GPU services + 6–12 agents, and (4) a fleet of anti-bluff traps where a green test hides a dead feature. Those four are the spine of the register below.

PASS 1 — Goal Verification (does the proposed architecture achieve the mandate?)

Mandate decomposed (from 00_programme_master_plan.md §1-2): extend HelixLLM → local models on RTX 5090 → expose via HelixAgent to HelixCode/CLI agents (OpenAI + Anthropic REST) → near-frontier coding/agent quality at 6–12 concurrency → add vision, generative (image/video WAN+LTX/vector), translation, embeddings/RAG/HelixMemory, MCP/ACP, Whisper/Tesseract, codegraph/opendesign → all rootless-podman via containers submodule → LLMsVerifier + claude_toolkit integration → setup/PATH → full test + Challenges + HelixQA → keep running for live operator testing.

Coverage verdict — every capability HAS a researched, cited landing place:

Mandate element	Covered by	Verdict
Local coding/agent model, 6–12 concurrency	01 (Qwen3-Coder-30B-A3B MoE, vLLM/llama-server, KV math)	covered, on-card re-measure pending
Vision VLM	02 (Qwen3-VL-8B/30B-A3B)	
Image gen	02 (FLUX.1 fp8, ComfyUI + OpenAI bridge)	
Video gen WAN + LTX (mandatory)	02 (LTX-2 fast + WAN 2.2 14B fp8)	both fit 32 GB
Vectorize pixel → SVG	02 (vtracer + StarVector-8B + potrace)	
Translation local + cloud	03 (NLLB/MADLAD CT2 + TOWER+ + DeepL/Google/Azure/AWS)	

Mandate element	Covered by	Verdict
Embeddings + RAG + HelixMemory	04 (Qwen3-Embed-4B + BGE-M3 + Qdrant + Zep/mem0)	
MCP + ACP + tool-calling + OKF	05 (MCP Go SDK, Zed ACP, JSON-Schema+constrained decoding, OKF=content)	w/ clarifications
Provider coverage (Poe/Perplexity/Sakana/Xiaomi/Tencent/...)	06 (adapter matrix)	(some parked/blocked)
Whisper STT + Tesseract OCR	07 (faster-whisper/Parakeet + Tesseract 5.5 tiers)	
codegraph + opendesign core providers	08 (wiring-fix + CodeIntel interface + token adapters)	
Rootless-podman via containers submodule	09 (boot/compose/health API mapped)	
LLMsVerifier extension (local + new providers, CONST-040)	10 (probe design + fail-closed flags)	
claude_toolkit HelixAgent provider + aliases	11 (PATH-detect → resolved-record path)	
HelixQA vision + full test matrix	12 (9 capability banks + self-validated analyzers)	

Goals NOT (fully) covered — the genuine research holes:

- G-GAP-A — HelixLLM canonical API contract is unconfirmed at the capability level.** Two reports disagree: the inventory (00_codebase_inventory §1) FOUND HelixLLM serving OpenAI + Anthropic surfaces; but stream 12 (§Part-1 honesty boundary + Risk 1) could NOT locate a canonical HelixLLM service spec (endpoint paths / request-response schemas / which capabilities actually ship) and marks every HelixQA http/json: path a # CONFIRM placeholder. **Net: the transport exists; the exact per-capability contract the tests and the gateway must bind to is UNCONFIRMED.** This blocks HelixQA bank authoring and the gateway's per-capability routing until the real surface is

read/confirmed. (Owner: Phase-P task 0 — read the HelixLLM API surface before writing any bank/gateway route.)

- **G-GAP-B — the VRAM residency broker / model-scheduler is a NEW component with no design depth yet.** The ADR (Decision 3) and streams 01/02/04/09 all name it as required, but no report designs its admission algorithm, eviction policy, request-class queue, or its interaction with vLLM sleep-mode / Ollama keep-alive at the code level. It is the single largest **new-build** in the programme and currently only a sketch. (Owner: dedicated Phase-1/3 design item.)
- **G-GAP-C — HelixLLM is not a pinned own-org submodule at the meta-repo root** (inventory §1 GAP): two on-disk checkouts (submodules / helix_llm + dependencies / HelixDevelopment / helix_llm) with no root .gitmodules entry → version-drift risk + codegraph double-indexing. A structural gap the plan must fix before building on it.
- **G-GAP-D — end-to-end PATH-install / setup story is absent.** Inventory §9 GAP: “no single top-level target builds llama.cpp → helix_llm → helix_agent → CLI aliases end-to-end.” Mandate 2.E requires it; no report delivers the install script design.
- **G-GAP-E — driver-version decision unresolved.** Host is driver 570.169 (host baseline); stream 01 + ADR recommend ≥575 for sm_120 (vLLM working config cites 575.64.03). Whether 570.169 suffices or a host driver upgrade is required is **UNCONFIRMED** and is an operator/ host action, not a code task. (See RZ/CX Pass-4.)
- **Parked operator clarifications (do not block, but scope 4 items):** GPT SOL, Google OKF role, which ACP (Zed strongly implied), Subquadratic (blocked-until-GA) — 03_open_clarifications.md.

Pass-1 conclusion: architecture achieves the mandate on paper; five real gaps (A–E) must be closed in Phase-P before implementation is grounded. None is a dead end; A and B are the two that most change the plan.

PASS 2 — Regression / Blast-Radius (what EXISTING working behavior could this break?)

Every touch point below is real code the programme WILL modify. Each is a §11.4.92-Pass-2 contract that must be preserved (proven by a runtime signature per §11.4.108, not assumed).

#	Existing working behavior	Where	How the programme touches it
BR-01	HelixLLM scored provider-fallback chain, llamacpp always last resort	helix_llm cmd/helixllm/main.go:327	GPU wiring + new local model registry changes llama server launch config
BR-02	HelixAgent ~45 provider adapters + provider registry	helix_agent internal/llm/providers/*, services/provider_registry.go	Adding stream-06 providers + local HelixAgent-a provider + CONST-040 fields
BR-03	LLMsVerifier single-source-of-truth (CONST-036/037) drives BOTH HelixLLM fallback scoring AND HelixAgent InitializeFromStartupVerifier	llms_verifier; helix_llm main.go:328; helix_agent provider_registry.go:588	Demote static registry flags → probe-derived; add RAG/Skills/Plugins columns + migration
BR-04	claude_toolkit provider aliases (env-key-driven resolution)	claude_toolkit scripts/claude-providers.sh, providers_resolve.py	Add PATH-detect for HelixAgent WITHOUT polluting the pure resolver
BR-05	claude_toolkit vendored LLMsVerifier (1 commit behind origin/main)	claude_toolkit submodules/LLMsVerifier	Bump + re-incorporate to latest
BR-06	.mcp.json MCP servers (codegraph, media-validator enabled; opendesign disabled)	root .mcp.json	Rewrite codegraph entry (drop

#	Existing working behavior	Where	How the programme touches it
			macOS path), enable opendesign
BR-07	codegraph index (6.09 GB / 102,708 files, live on host)	.codegraph/, PATH binary 1.1.1 vs npm 1.2.0	Re-scope excludes (\$11.4.79) + reconcile version drift re-index
BR-08	HelixQA existing banks + anti-bluff engine (ContentAssertingResolver, conduit)	helix_qa pkg/testbank, pkg/conduit	Add 9 new HelixLLM banks + per-cap analyzer
BR-09	HelixAgent OpenAI/Anthropic REST server (the model-list surface Claude Code reads)	helix_agent internal/handlers/openai_compatible.go	Extend as the unified /v1 gateway (AD Decision 4) + add capabilities block to /v1/models
BR-10	HelixCode CLI HELIX_LLM_PROVIDER / HELIX_LLM_USE_LLAMACPP env wiring	helix_code/cmd/cli/main.go, helix_agent providers/helixllm	Repoint at new local serving

Pass-2 conclusion: the highest-blast-radius touch points are **BR-01 (fallback floor)**, **BR-03 (verifier SoT feeds two consumers)**, **BR-06/BR-09 (the surfaces Claude Code/HelixCode auto-recognize)**. All are additive-safe if the discipline is: additive struct/JSON changes, fail-closed defaults, and a runtime signature proving the OLD contract still holds on a clean deploy (\$11.4.108).

PASS 3 — Cross-Feature Interaction (shared state / timing / hardware contention)

The programme's defining property: **one host, one 32 GB GPU, ~10 GPU-hungry services, 6–12 concurrent agents**. Everything contends.

3.1 Single-GPU VRAM contention (the master cross-feature risk — CX-02)

- **Fact (ADR Decision 3 + 01 §4 + 02 §7 + 04 §8 + 09 §3):** 32 GB CANNOT co-host a 30B coder + 30B VLM + FLUX + WAN 2.2 simultaneously. Aggregate resident VRAM » 32 GB.
- **Time-slicing under CDI has NO memory isolation** (09 §3.1, cited): two services each grab 20 GB → one OOMs. MIG is datacenter-only (not on GeForce 5090) → ruled out.
- **The interaction chain:** coding-agent fleet (always-resident, ~18 GB Q4 + KV) ↔ embedder (~8 GB or drop to 0.6B) ↔ on-demand VLM/FLUX/WAN/TOWER+. If a video render fires while 12 agents hold KV, the burst OOMs the fleet — a user-visible outage of the primary path.
- **Mitigation (converged across streams):** tiered residency (always-resident ≤ ~6 GB; one warm model; burst = single-owner) + explicit per-engine caps (--gpu-memory-utilization, CUDA_MPS_PINNED_DEVICE_MEM_LIMIT, OLLAMA_GPU_OVERHEAD) + **the VRAM budget-broker (G-GAP-B) + §11.4.119 single-resource-owner** serialization of burst classes + GPUHealthCheck VRAM floor gating each start (09 §1.4/§3.2).

3.2 Concurrency (6–12 agents) vs on-demand model-swap latency

- **Interaction:** the fleet wants low TTFT under load; on-demand VLM/gen wants VRAM the fleet is holding. A cold model reload is seconds-to-minutes; **vLLM sleep-mode wake is 18–200× faster than cold reload** (09 §3.1) and is the lever that makes swap tolerable. Ollama keep-alive is the fallback.
- **Danger:** if the broker evicts the resident coder to serve a one-off vision request, all 12 agents stall on the reload → a latency cliff. **Policy must protect the always-resident fleet model from eviction** and serialize burst classes instead (§11.4.119).
- **KV math is the capacity gate (01 §4):** MoE Qwen3-Coder-30B-A3B (~⅓ KV of a dense 32B) is what makes 12 @16k feasible; a dense 32B tops out at ~4 @16k. Choosing the dense model for the fleet silently caps concurrency at 4 — a cross-feature trap.

3.3 Port / endpoint conflicts (shared host namespace)

Observed/claimed ports across reports: HelixAgent : 8100 (11), HelixLLM native : 8443 (11) + llama-server : 8080/: 8081 (01), vision : 8090 (02), OpenDesign daemon : 7456 (08), Qdrant : 6333 (inventory §1.4), STT : 8000 (07), vLLM : 8000 (01/02). **:8000 collides between vLLM and the STT service; :8080/:8090 are reused across capability examples.** Under one rootless-podman pod network these must be centrally allocated. (Owner: a single port-map ADR in Phase-P; the unified /v1 gateway (ADR Decision 4) hides most of them behind one surface, but the backing containers still need distinct host/pod ports.)

3.4 Shared config / secrets

- **Secrets sprawl:** HelixAgent HELIXAGENT_API_KEY, HelixLLM HELIX_LLM_API_KEY, OpenDesign HELIX_OD_BYOK_*, per-cloud-provider keys, HF token (gated weights), translation cloud keys — all must live in .env (0600, gitignored, CONST-042/§11.4.10), never in .mcp.json / compose / argv. Cross-feature risk: one leaked key in a committed compose file or a proof artefact. Guard: the §11.4.109 PreToolUse guard + secret-redactor in every proof capture (already present in claude_toolkit tests, 11 §1.6).
- **Config single-source:** LLMsVerifier is the model-metadata SoT for BOTH HelixLLM and HelixAgent — a config change (endpoint map, freshness policy) ripples to both (BR-03).

3.5 codegraph index bloat vs §11.4.79

- **Fact (08 §1.1):** the live index is 6.09 GB / 102,708 files because it indexes vendored third-party under submodules/helix_qa/tools/opensource/** and the duplicate dependencies/HelixDevelopment/helix_llm/** — **violating §11.4.79(b)** (third-party must be EXCLUDED) and polluting RAG hits with appium/docling/etc.
- **Interaction with the RAG stack (04):** if codegraph is fused into the retriever (04 §4.2, 08 §5.1) while the index is bloated, agent context gets polluted → worse answers, more tokens (defeats §11.4.141). Excludes (08 §4.2) must land BEFORE codegraph is wired as a RAG source.

3.6 CUDA-graph / FlashAttention shared assumption

Every torch-based engine (vLLM, ComfyUI, TEI, CTranslate2-GPU) shares the SAME toolchain constraint (ADR Decision 1): CUDA 12.8, torch 2.9 cu128, **FA2 not FA3** (VLLM_FLASH_ATTN_VERSION=2). A single mismatched image silently falls back to a slow path or fails to load FP8/FP4 kernels. One pinned Dockerfile.cuda-base is the shared dependency; a drift in it breaks every capability image at once (blessing and curse).

Pass-3 conclusion: the cross-feature risks are dominated by **VRAM contention + swap latency + the broker that must arbitrate them** (3.1/3.2), then **port allocation** (3.3) and **index bloat feeding RAG** (3.5). These are not independent — the broker (G-GAP-B) is the keystone that, done wrong, cascades into the fleet-outage failure mode.

PASS 4 — Deep-Research Validation Gaps (every UNCONFIRMED → must-verify-before-commit)

Per §11.4.6 nothing UNCONFIRMED may be treated as fact at commit. Each row = a gate the implementation MUST pass with captured evidence before the dependent work proceeds.

ID	UNCONFIRMED claim	Report	Verification command / proof (captured-evidence gate)
V-01	vLLM/PyTorch prebuilt wheels work on sm_120	01 §3, 02 §7	Build from source (torch 2.9 cu128, TORCH_CUDA_ARCH_LIST=12.0, FA2); prove with <code>vllm serve ... && curl /v1/chat/completions</code> returning real output + <code>nvidia-smi</code> VRAM delta
V-02	llama.cpp CUDA build for sm_120	01 §5B, host G-HOST-3	<code>cmake -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=120 && cmake --build</code> ; then a real inference proof (tok/s + VRAM delta), NOT build-success-only (§11.4.108)
V-03	ALT-Linux packages nvidia-container-toolkit	09 §2.1 R1, host	<code>nvidia-ctk --version</code> after install; if absent, NVIDIA rpm repo or from-source <code>nvidia-ctk</code> . This gates ALL GPU work — verify FIRST.
V-04	Rootless CDI GPU passthrough actually works here	host G-HOST-1, 01 §7, 09 §2.4	<code>podman run --device nvidia.com/gpu=all ... nvidia-smi</code> prints the RTX 5090 (podman #17539 rootless-fails-where-privileged-works risk)
V-05	Driver 570.169 sufficient vs ≥575 recommended	host baseline, ADR Decision 1, 01	Attempt the V-01/V-02 builds on 570.169; if kernels fail, host driver upgrade decision → operator (G-GAP-E)

ID	UNCONFIRMED claim	Report	Verification command / proof (captured-evidence gate)
V-06	CTranslate2 ships sm_120 GPU int8 kernels (prebuilt)	03 §2.2/§7, negative finding	Build CT2 in the CUDA-12.8 image; assert GPU int8 inference visible in nvidia-smi at image-build; else CPU-int8 fallback
V-07	podman-compose (pip) honors CDI devices: where podman compose drops them	09 GAP-2 (cited podman #28436)	Preflight-assert detectComposeCmd() == podman-compose; boot a CUDA container via compose, assert nvidia-smi sees the GPU
V-08	Official Go MCP SDK ships a Streamable-HTTP server transport	05 §1.3	Read go-sdk/mcp package docs + a spike server before committing the remote-gateway design; else mark3labs/mcp-go fallback
V-09	No first-party Go ACP SDK exists	05 §2.4/§7, 08	Hand-roll the ACP v1 JSON-RPC-over-stdio surface in Go (LSP-sized) OR bridge TS/Rust; budget the work
V-10	Benchmark tok/s (Qwen3-Coder ~140, dense 32B ~45–58, whisper RTFx ~33×, etc.)	01, 02, 03, 07 (all “estimate/aggregator”)	Re-measure EVERY headline number on the actual 5090 with our fixtures before any SLA/claim (§11.4.6/ §11.4.107(13))
V-11	2026 model names GLM-4.7/5, Qwen3.5/3.6, DeepSeek V4	01 §2 (UNCONFIRMED)	Excluded from recommendations; do NOT plan around them until a first-party model card appears
V-12	Qwen3-Coder-30B-A3B exact KV-head count (drives concurrency math)	01 §4 (approx)	Read the model config.json; recompute the KV table before sizing the fleet
V-13	Model-card specs behind ¹ (Qwen3-Embedding dims/ctx/MTEB, Stella, gte)	04 §1.2	Verify each against the model card before pinning dims/context
V-14		04 §5	

ID	UNCONFIRMED claim	Report	Verification command / proof (captured-evidence gate)
	LongMemEval 63.8 vs 49.0 (Zep vs mem0)		Verify against the primary Zep/Graphiti paper before treating as canonical
V-15	COMET/BLEU as quality proof (contamination)	03 §1.3, 12 (d)	Gate professional-quality claims on QE + human/MQM spot-check, never COMET alone
V-16	Hyperbolic / Baseten base URLs (3rd-party sourced)	06 GROUP-E	Fetch official docs before writing those two adapters
V-17	Parakeet-TDT vs WhisperX word-timestamp accuracy	07 negative finding	Verify against NeMo docs before using Parakeet timings for the subtitle oracle
V-18	BabelDOC / TranslateGemma / Tower-scaling future-dated arXiv ids	03 sources	Treat as UNCONFIRMED; do not cite their exact numbers
V-19	codegraph MCP tool set + .mcp.json shape on THIS host	08 §2 (probed 1.1.1 vs npm 1.2.0)	Reconcile version drift, then codegraph_validate.sh cross-submodule probe
V-20	HelixLLM per-capability API contract	12 Risk-1, G-GAP-A	Read the real HelixLLM endpoint/schema surface; fill every # CONFIRM before authoring banks/gateway routes

Pass-4 conclusion: the toolchain cluster **V-01...V-07 is the critical-path validation front** — none of the AI stack runs until they pass, and **V-03 (ALT-Linux toolkit packaging) is the very first unknown to resolve** because it gates the CDI setup that gates everything. V-10 (re-measure all benchmarks) and V-15/V-20 (metric + contract honesty) are the anti-bluff front.

PASS 5 — Anti-Bluff Surface (where could a test/gate PASS while the feature is broken?)

Each row: the bluff, the report that flagged it, the §-mitigation.

AB-#	Bluff (green test, dead feature)	Flagged by	Mitigation (mechanical)
AB-01	Frozen/stale-frame vision — one captured frame “shows a picture” but it’s a stuck decoder / previous content	02, 12 (b/c), §11.4.107	Freeze-detection + frame-advance oracle + not-stale cross-check; ffp _{probe} -count_frames; self-validated analyzer golden-good/bad
AB-02	Silence-hallucination STT — Whisper invents words on silence → “expected words present” PASSES on nothing	07 §5.1, §11.4.68	Parakeet (no silence-hallucination) for the assertion path; VAD + no-speech-prob gating for Whisper; qa-audio-probe RMS floor first; self-validated audio fixtures
AB-03	Static-registry capability flags — CONST-040 MCP/ACP/LSP set from a literal, not a probe	10 §1.4 Risk-2, CONST-040	Demote registry.go to seed-only; probe-derived flag + CapabilityEvidence; no evidence ⇒ false (fail-closed) ; blocked on adding Message.ToolCalls field (10 §2.4)
AB-04	PATH-present ≠server-running — command -v helixagent succeeds → alias marked “verified” while :8100 is down	11 §Risk-2, §4.a	PATH gate registers alias but model-enum + verdict come from a live /v1/models + real chat probe; server-down ⇒ honest unverified
AB-05	LLMsVerifier bump-without-rebuild — gitlink bumped, .local-cache/bin binaries stale → toolkit tests STALE code (SOURCE → ARTIFACT)	11 §Risk-3, §2.3, §11.4.108	Force-rebuild code-verification + model-verification; re-run verify_providers_live.sh before committing the bump
AB-06	podman-compose drops CDI devices silently — podman compose starts the container with NO GPU, health passes on the port	09 GAP-2 (podman #28436)	Pin podman-compose (pip); preflight-assert the resolved backend; GPUHealthCheck VRAM-floor + in-container nvidia-smi gate
AB-07	Metric-gaming (COMET/MTEB) — COMET/BLEU	03 §1.3, 04 §8, 12 (d)	QE (CometKiwi) + human/MQM spot-check gate; back-

AB-#	Bluff (green test, dead feature)	Flagged by	Mitigation (mechanical)
	green while human quality is not (Tower-v2-70B #1 COMET, lost 9/11 human)		translation metamorphic; per-langpair quality ledger
AB-08	codegraph index that lies — own-org submodule silently excluded → agent “can’t find” a symbol that exists	08 §4.2, §11.4.79	Cross-submodule symbol probe in <code>codegraph_validate.sh</code> + paired mutation (exclude → validate MUST FAIL → restore)
AB-09	Analyzer-is-the-bluff — CLIP/COMET/WER/faithfulness analyzer miscalibrated or lenient PASSes broken output	12 Risk-2, §11.4.107(10)	Mandatory golden-good + golden-bad + paired §1.1 mutation on EVERY analyzer; thresholds calibrated on OUR fixtures, never literature
AB-10	Build-success ≠ works — image builds, but kernels don’t load on sm_120 / feature dead on clean deploy	ADR Decision 1, 01, §11.4.108	Per-image runtime proof (real inference/embed/transcribe + <code>nvidia-smi delta</code>) at build; runtime-signature on clean deploy
AB-11	File-exists ≠ correct — HelixQA GlobEvidenceResolver passes on <code>echo stereo > codec.txt</code>	12 §1.2	ContentAssertingResolver grammar (<code>json:/match:/min_int:)</code> over the verdict artefact; bare nonempty forbidden for a correctness claim
AB-12	Subtitle chrome-as-dialogue — OCR reads a menu label as a caption	07 §5.2, §11.4.137	Chrome-label denylist + position-band + <code>cadence ≥ 2</code> + fuzzy-match vs source cue; honest SKIP on FLAG_SECURE black frames
AB-13	Verdict-without-evidence — a PASS <code>challenge_verdict</code> with no preceding <code>evidence_captured</code>	12 §1.4, §11.4.116	Conductor treats a PASS with no backing <code>evidence_captured</code> as a contradiction ⇒ FAIL
AB-14	Freshness stamped without a run —	10 §2.5	Timestamps come from real <code>CompletedAt</code> of a real probe;

AB-#	Bluff (green test, dead feature)	Flagged by	Mitigation (mechanical)
	VerifiedAt=time.Now() with no probe		24h/60s policy bound to actual runs

Pass-5 conclusion: the anti-bluff machinery to catch all 14 largely **already exists** (HelixQA ContentAssertingResolver + conduit, LLMsVerifier evidence-backed flags, claude_toolkit proof capture, §11.4.107 liveness). The programme’s job is to **use it, not weaken it** — the two NEW hazards are **AB-03 (fail-closed capability flags, blocked on Message.ToolCalls)** and **AB-06 (the silent no-GPU compose backend)**, both of which need a mechanical gate, not vigilance.

CONSOLIDATED DANGER-ZONE / BOTTLENECK REGISTER

Severity: **Critical** = blocks the whole programme or causes a user-visible primary-path outage / a §11.4 PASS-bluff. **High** = blocks a capability or a major regression. **Med** = localized.

ID	Title	Sev	Streams that flagged	Blast radius	Mitigation
DZ-01	Rootless GPU passthrough does NOT exist yet (nvidia-ctk + CDI absent)	Critical	host, 01, 02, 03, 04, 07, 09	ALL GPU work — nothing serves until fixed	§2 of stream 09: cgroups → cdi nvidia-smi pr packaging FIRS
DZ-02	Blackwell sm_120 toolchain fragility (wheels fail; source build required)	Critical	01, 02, 03, 04, 09, ADR CX-01	Every GPU image (vLLM/ llama.cpp/ ComfyUI/TEI/ CT2)	One pinned Doc (CUDA 12.8, toro image runtime y
DZ-03	32 GB VRAM contention across ~10 services + 6–12 agents	Critical	01, 02, 04, 09, ADR CX-02, RZ-03	Primary coding-agent path (fleet OOM)	Tiered residency + per-engine cap + GPUHealthCh
DZ-04	VRAM residency broker is a NEW, under-designed component	Critical	ADR Decision 3, 01, 02, 04, 09	Arbitrates DZ-03; wrong policy → fleet-eviction latency cliff	Dedicated design class + VRAM bu from eviction; v
DZ-05		High			

ID	Title	Sev	Streams that flagged	Blast radius	Mitigation
	HelixLLM per-capability API contract UNCONFIRMED		12 Risk-1, inventory (partial)	HelixQA banks + gateway routing blocked	Phase-P task 0: n surface; fill even
DZ-06	LLMsVerifier static capability flags (CONST-040 bluff) + missing Message.ToolCalls	High	10 §1.4/ Risk-1/Risk-2	Both HelixLLM + HelixAgent consume the SoT	Add ToolCalls (change); probe+
DZ-07	podman-compose backend silently drops CDI GPU devices	High	09 GAP-2	Any GPU service booted via wrong backend → no-GPU, green health	Pin podman - con backend; upstre #3
DZ-08	codegraph index bloat (6 GB/102k) indexes third-party (violates §11.4.79)	High	08 §1.1/§4.2	RAG-fused agent context polluted; slow re-index	Exclude patches validate probe + 1.1.1 ↔ 1.2.0
DZ-09	HelixLLM not a pinned root submodule (2 checkouts, drift)	High	inventory §1, 08	Version drift + double-index	Add root .gitm canonical check
DZ-10	claude_toolkit LLMsVerifier bump-without-rebuild (SOURCE → ARTIFACT)	High	11 §Risk-3	Toolkit exercises stale verifier	Force-rebuild b before gitlink b
DZ-11	Frozen-frame / silence-hallucination / analyzer-bluff family	High	02, 07, 12, ADR RZ-05	Every media/ vision/STT test could pass-bluff	§11.4.107/.137/.1 golden-good/ba
DZ-12	.mcp.json host-rot (macOS paths; opendesign disabled)	Med	05, 08	Claude Code can't launch codegraph/ opendesign; a malformed edit breaks all MCP	Portable bare-c injected BYOK; I unchanged; JSO
DZ-13	Port/endpoint collisions (: 8000	Med	01, 02, 07, 08, 11		One port-map A backends behin

ID	Title	Sev	Streams that flagged	Blast radius	Mitigation
	vLLM ↔ STT; reused :8080/8090)			Boot failures / silent wrong-service	
DZ-14	Metric-gaming (COMET/MTEB/BLEU green ≠quality)	Med	03, 04, 12, ADR CX-05	False “professional quality” claims	QE + human/MC (back-translation)
DZ-15	Large weights unversioned → fresh clone can’t run (§11.4.77)	Med	02, 03, 04, 07, 09, ADR CX-07, RZ-07	Reproducibility / fresh-clone build	fetch_weights*.yaml per weight pre-build stamp
DZ-16	Go SDK gaps (MCP Streamable-HTTP UNCONFIRMED; no first-party Go ACP SDK)	Med	05 §Risk-1	MCP remote gateway + ACP editor-recognition	Verify go-sdk/ in Go; pin version
DZ-17	License traps (Jina reranker/ embeddings CC-BY-NC)	Med	04 §8, ADR CX-06	Shipping non-commercial weights	Ship only Apache/ nomic/mxbai) s
DZ-18	Driver 570.169 vs ≥575 recommended	Med	host, ADR, 01, 04	May block sm_120 kernels	Attempt builds decision → oper
DZ-19	Host safety while driving GPU hard (thermal/power §11.4.133)	Med	00 RZ-08, ADR	Host/hardware safety	Thermal-aware evidence under unverified GPU-
DZ-20	Operator-ambiguous provider names (GPT SOL/OKF/ACP/SubQ)	Med	06, 05, clarifications	Wasted cycle building the wrong thing	Batched operator SubQ blocked-u
DZ-21	End-to-end setup/ PATH-install absent (G-GAP-D)	Med	inventory §9	Mandate 2.E unmet; operator can’t install the stack	Design a top-level llama.cpp → heli
DZ-22	Poe points-billing (not per-token) + verify-before-code adapters (Hyperbolic/Baseten)	Low	06	Adapter surprises	Treat Poe as OA docs for Hyperb

CRITICAL-PATH / SEQUENCING ANALYSIS

The programme has a **hard serial prefix** — a chain where each link is impossible until the prior link is proven with captured evidence (§11.4.108). Get the order wrong and every downstream task builds on sand.

- [0] HOST PREREQ — GPU passthrough (DZ-01, blocks EVERYTHING GPU)
 - ├ V-03 Install nvidia-container-toolkit on ALT-Linux (verify nvidia)
 - ├ nvidia-ctk config --set ...no-cgroups (rootless)
 - ├ nvidia-ctk cdi generate → ~/.config/cdi/nvidia.yaml
 - └ V-04 PROOF: podman run --device nvidia.com/gpu=all ... nvidia-smi p
↓ (nothing below runs until this prints)
- [1] TOOLCHAIN BASELINE (DZ-02) — pinned Dockerfile.cuda-base (CUDA 12.8, t)
 - ├ V-05 decide driver 570.169 vs upgrade ≥575
 - ├ V-01 build vLLM from source for sm_120 → real /v1/chat/completio
 - ├ V-02 build llama.cpp -DCMAKE_CUDA_ARCHITECTURES=120 → real tok/s
 - └ V-07 pin podman-compose (pip); assert CDI devices honored (DZ-07)
↓
- [2] PRIMARY SERVING CORE (DZ-03/04 begin)
 - ├ V-12 confirm Qwen3-Coder-30B-A3B KV heads → recompute concurrency
 - ├ llama-server / vLLM serving the fleet model on GPU (real inf
 - ├ VRAM broker DESIGN (G-GAP-B) — tiered residency, protect fle
 - └ HelixLLM GPU launch wiring (BR-01 preserve fallback floor)
↓
- [3] GATEWAY + VERIFIER (DZ-06)
 - ├ add Message.ToolCalls (unblocks tool-call/MCP probes)
 - ├ LLMsVerifier local + new-provider probes, fail-closed CONST-
 - ├ HelixAgent unified /v1 gateway + /v1/models capabilities blo
 - └ DZ-09 pin HelixLLM as root submodule
↓
- [4] EXTENDED CAPABILITIES (each on-demand via broker; parallelizable AFTER)
 - ├ vision (Qwen3-VL) ├ image-gen (FLUX) ├ video (LTX/WAN) ├
 - ├ translation (NLLB CT2 + TOWER+ ; V-06 CT2 sm_120) ├ e
 - └ Whisper STT + Tesseract OCR (DZ-11 self-validated analyzers)
↓
- [5] INTEGRATIONS — codegraph/opendesign wiring fix (.mcp.json DZ-12) + ex
+ MCP/ACP (DZ-16 hand-roll Go ACP)
↓
- [6] claude_toolkit (DZ-10 rebuild-before-bump) + setup/PATH install (DZ-21)
↓
- [7] FULL TEST — HelixQA 9 banks (DZ-05 confirm contract first) + Challenge
↓
- [8] LIVE RUN + resumption doc (§11.4.131)

Hard dependency rules (violating any = building on sand): 1. **Nothing GPU exists until [0] prints `nvidia-smi` in a rootless container.** DZ-01 is the universal gate. **V-03 (ALT-Linux toolkit packaging) is the first unknown to resolve.** 2. **No capability image is trusted until [1] proves a real inference on the card** — build success is not acceptance (§11.4.108/AB-10). 3. **The VRAM broker ([2] design) must exist before ANY extended capability ([4]) is loaded on-demand** — otherwise the first VLM/gen request OOMs the fleet (DZ-03/04). 4. **Message.ToolCalls ([3]) blocks truthful CONST-040 tool/MCP flags** — do it early. 5. **codegraph excludes (DZ-08) must land before codegraph is fused into RAG ([4] embeddings)** — else the RAG context is polluted. 6. **The HelixLLM contract (DZ-05) must be confirmed before HelixQA banks ([7]) are authored** — guessing it is itself a bluff.

TOP 10 THINGS MOST LIKELY TO GO WRONG (and how each is caught EARLY)

1. **ALT-Linux has no `nvidia-container-toolkit` package (V-03/DZ-01).** → Caught at task 0 by `nvidia-ctk --version` failing; fallback = NVIDIA rpm repo or from-source `nvidia-ctk`. *If not caught early, every GPU task in the plan is blocked and the schedule is fiction.*
2. **Rootless CDI resolves privileged but not rootless (podman #17539) (V-04/DZ-01).** → Caught by the in-container `nvidia-smi` proof BEFORE any stack boot; fallback path documented.
3. **vLLM/llama.cpp source build fails on driver 570.169 (V-05/DZ-02/DZ-18).** → Caught at [1] by the real-inference proof; triggers the operator driver-upgrade decision immediately, not after the whole stack is wired.
4. **First on-demand VLM/gen request OOMs the 12-agent fleet (DZ-03/04).** → Caught by the GPUHealthCheck VRAM floor refusing the start + a stress bank (LLM-CONCURRENCY-001 + a burst-during-load chaos test §11.4.85) BEFORE it hits a user.
5. **podman compose silently starts GPU containers with NO GPU (AB-06/DZ-07).** → Caught by the preflight backend-assert (`detectComposeCmd() == podman-compose`) + in-container `nvidia-smi`; a green port-health is NOT accepted as GPU proof.
6. **CONST-040 flags stay literal-sourced because Message.ToolCalls is missing (AB-03/DZ-06).** → Caught by the fail-closed rule (no evidence = false) + the analyzer self-validation golden-bad fixture; the field is scheduled as the first [3] change.
7. **claude_toolkit bump ships stale verifier binaries (AB-05/DZ-10).** → Caught by the §2.3 force-rebuild + `verify_providers_live.sh` proof-capture gate BEFORE the gitlink commit.

8. **codegraph re-index pollutes RAG with vendored third-party (AB-08/DZ-08).** → Caught by the `codegraph_validate.sh` cross-submodule probe + a DB-size budget + the paired mutation, run before codegraph is wired as a retriever.
 9. **A media/STT/VLM analyzer is itself the bluff (AB-01/02/09/DZ-11).** → Caught by the MANDATORY golden-good + golden-bad + paired §1.1 mutation on every analyzer, in the meta-test — a lenient analyzer FAILs its own golden-bad.
 10. **HelixLLM API contract guessed wrong → banks/gateway routes bind to a non-existent surface (DZ-05).** → Caught by making “read the real HelixLLM API surface” the FIRST Phase-P task; every dependent # CONFIRM placeholder is a hard gate that cannot be closed by guessing.
-

Honest boundary (§11.4.6)

This synthesis reduces the *known* risk surface; it does not prove the programme will succeed. The four Critical zones (DZ-01...04) are all **host/hardware-empirical** — their true state is knowable ONLY by running the V-01...V-07 proofs on THIS card, which has not happened. Until then the toolchain, passthrough, and VRAM-broker viability are UNCONFIRMED in the strict sense. The register is complete against the 13 input reports; a capability or constraint not surfaced by any of the 12 streams is, by construction, not here — the §11.4.118 discovery-pass at implementation time remains mandatory.

Sources

All claims trace to the 13 prior reports under `docs/research/07.2026/` (`00_master/{00_programme_master_plan, 00_codebase_inventory, 01_host_baseline_evidence, 02_cross_cutting_foundations_ADR, 03_open_clarifications}, 01_local_models_serving, 02_vision_generative, 03_translation, 04_embeddings_rag, 05_mcp_acp_protocols, 06_providers_coverage, 07_stt_ocr_whisper_tesseract, 08_codegraph_opendesign, 09_containers_deployment, 10_llmsverifier_helixagent, 11_claude_toolkit, 12_helixqa_testing`), each of which carries its own cited web sources + captured host evidence. No new web research or source mutation was performed in this synthesis pass (§11.4.6).